11 雜湊表

Part III Data Structures

本章定位

Hash table以鍵值映射節省稀疏universe的空間，但速度依賴碰撞模型與負載。本章分開證明chaining、universal family、probe排列與兩層perfect hashing，並用Tombstone、去重和失敗query追蹤正確性，讓期望界與最壞保證各有明確條件。

前置知識

機率、模算術與平均情況分析。

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

本章路線

學習目標

- 從 direct-address 推到雜湊平均成本。
- 評估除法、乘法與 universal hashing 的假設。
- 分析 open addressing 的探測與完美雜湊。
- 11.1 Direct-address tables
- 11.2 Hash tables
- 11.3 Hash functions
- 11.4 Open addressing
- 11.5 Perfect hashing

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

11.1 Direct-address tables

Dictionary維護key→record的映射，本章先規定每個key至多一筆，insert同鍵時更新既有record。Universe U={0,⋯,M−1}夠小時，直接用T[key]，空槽為EMPTY，不與合法record混用。

Search讀T[k]、insert寫T[k]、delete清該槽，各最壞O(1)，空間Θ(M)。初始化每槽為EMPTY通常Θ(M)，不是因每次query O(1)就能把建表成本也寫成O(1)。位元集合可每key存一bit，但不包含satellite data。

例U=0..9，存在鍵2、5、8，只有T[2]、T[5]、T[8]指向record。這是用鍵的數值作索引，沒有碰撞。若鍵是長字串，讀完整key與轉換本身就需依長度付費；以下O(1)的hash與相等比較，採可在固定個word操作的鍵模型。

Lin Cing
https://github.com/xinhi6716/computer-science-self-s

11.2 Hash tables

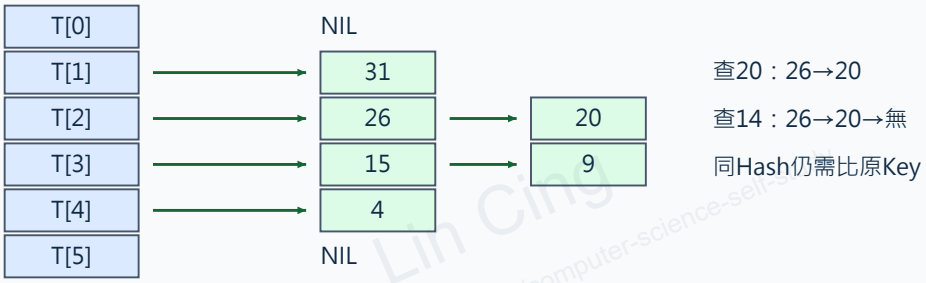
當|U|遠大於實際n筆，改用m槽和h:U→{0,⋯,m−1}。由鴿籠原理，|U|>m時任意固定h必存在不同鍵碰撞。Chaining把同槽記錄接成list，search仍必比原key，hash相同不是key相同。

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

11.2 Hash tables （續）

Chaining：槽與鏈結記錄是兩層存取

依序插4,9,15,20,26,31；m=6，h(k)=k mod6，頭插。



藍：槽；綠：Record。失敗查14仍掃槽2，不把Hash值當存在證明。

例m=6、h(k)=k mod6，依序插4,9,15,20,26,31且頭插。槽2為26→20，槽3為15→9，槽1為31、槽4為4。找20查兩個record，失敗查14也掃槽2的兩筆。這張特定表的成本不是對隨機h取期望的平均公式。

頭插O(1)以已知新key不存在為前提；本章dictionary更新語義要先search去重，故不能把整個insert都當無條件最壞O(1)。已持有有效handle且list為雙鏈時unlink O(1)，按key刪除仍要付search成本。

Chaining的兩種期望搜尋

令α=n/m，可大於1。Simple uniform hashing假設不同固定key的槽獨立均勻，hash計算O(1)。失敗query k不在表中，對每個已存x，I_x=[h(x)=h(k)]的期望1/m，掃描長度E[ΣI_x]=n/m=α，加固定索引成本得Θ(1+α)。

成功搜尋另假設每筆已存記錄等可能被問到。固定插入次序$x_1,\cdots,x_n$、頭插且沒有刪除，找x_i會檢查自己與之後插入的同槽記錄，所以

`E[成功檢查筆數] = (1/n)Σ_i(1+(n−i)/m) = 1+(n−1)/(2m)`。

這個較精確式需要平均query與頭插次序，不能把「每個槽等可能被成功查詢」當成理由：大chain含更多key，較可能被選到。若只需O(1+α)上界，可用包含該key的整條chain長度，不必採等機率成功query。

保住α=O(1)才有常數期望；任何固定表仍可能有一條n長chain，單次最壞Θ(n)。若擴表重選h，搬移和重新計hash的成本也要計，動態表攤銷另見第17章。

11.3 Hash functions

Division method為h(k)=k mod m。若m=2^p，只保留最低p位，例0與8、6與14在m=8時碰撞。選質數有時能避開特定規律，但不是對任意key集合的均勻性定理；選好的m也不能消除鴿籠限制。

長字串可用Horner逐字計算：$H_0=0$，$H_{i+1}=(B \cdot H_i+c_i) \bmod m$，得到完整base-B整數的mod m而不建立巨大整數，時間Θ(字元數)。若B≡1 mod m，任何字元重排有相同和，仍會碰撞，公式本身不保證分布好。

Multiplication method採0<A<1，$h(k)=\lfloor m \cdot \mathrm{frac}(kA) \rfloor$。若A=s/2^w、m=2^p，先算k・s的低w位r_0，再取r_0的最高p位，不能誤取整個2w位乘積的最高p位。

11.3 Hash functions （續）

Multiplication Hash：取低Word的高位

w=8, s=159, k=13；k·s=2067=8·256+19；m=16取p=4位。

15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
0	0	0	0	1	0	0	0	0	0	0	1	0	0	1	1

高Word 00001000 = 8　　低Word 00010011 = 19

橘色是低Word的高4位：0001 → h=1。

等價計算：(2067 mod256) >>4 = 19 >>4 = 1。

灰：捨棄的高Word；橘：取出的位；藍：低Word其餘位。

例w=8、s=159、k=13，乘積2067=8・256+19，低word=00010011。m=16取低word高四位0001，h=1；算⌊16・19/256⌋也得1。

Universal不是「看起來夠亂」

函數族H稱universal，如果對每對固定相異鍵x,y，都有Pr_(h均勻取H)[h(x)=h(y)]≤1/m。鍵集合與query必須固定且獨立於這次h的選擇；不是假設使用者key天生均勻。

對固定query k，碰撞對指示變數線性求和，若k不在表，E[chain長度]≤n/m；若在表則≤1+(n−1)/m。這只用pairwise碰撞界，完全不要求各碰撞事件互相獨立，也不聲稱每個槽的輸出都均勻。

因此初始化m槽之後，固定操作序列、最多O(m)筆同時存在時，chaining每次search及需去重的insert期望O(1)，總t次操作期望O(t)；初始化的Θ(m)仍另外計。若對手看到h或由回應自適應選新鍵，原本「固定鍵對」保證不能直接沿用。Universality也不是密碼學不可預測性，無須為此假定h是永久秘密。

質數Affine族的完整碰撞證明

選質數P大於所有合法key，a均勻取1..P−1、b取0..P−1，定義h_(a,b)(k)=((ak+b) mod P) mod m。

固定x≠y，先看r=(ax+b) mod P、s=(ay+b) mod P。因a、x−y皆在域F_P非零，r−s=a(x−y)≠0。反過來，每對r≠s唯一決定a=(r−s)(x−y)^(−1) mod P、b=r−ax mod P。因此均勻(a,b)對應均勻的P(P−1)個有序相異(r,s)。

固定r後，另P−1個可能s中，與r同mod m者至多⌈P/m⌉−1≤(P−1)/m，碰撞機率至多1/m，故H universal。若m≥P，mod m根本不再合併r,s，碰撞機率0，也成立。

需要P為質數是為了每個非零x−y可逆；任意合數不能照搬證明。運算成本假設P、乘積模運算與key都在指定word-RAM可O(1)處理的範圍；任意精度大整數須另外計費。

11.4 Open addressing

所有record直接放m槽內，故n≤m。每key有固定probe序列h(k,0),⋯,h(k,m−1)，同key搜尋/插入必重現同序列。為保證有空槽就可插，序列須覆蓋每槽，常要求是全表排列。

槽有三態：EMPTY（從本次建表起未曾使用）、DELETED（tombstone，曾刪除）、OCCUPIED(record)。Search遇key相等回傳；遇EMPTY可判不存在；遇DELETED繼續，最多m次探測。全表沒有EMPTY時仍會終止，不得用無限while等空槽。

EMPTY可提早停的理由是：若key曾被插到更後面，插入時不可能跳過當時EMPTY；後續刪除只變DELETED而不變EMPTY，所以沒有無故新生的停止缺口。

Tombstone與不重複Key的Insert

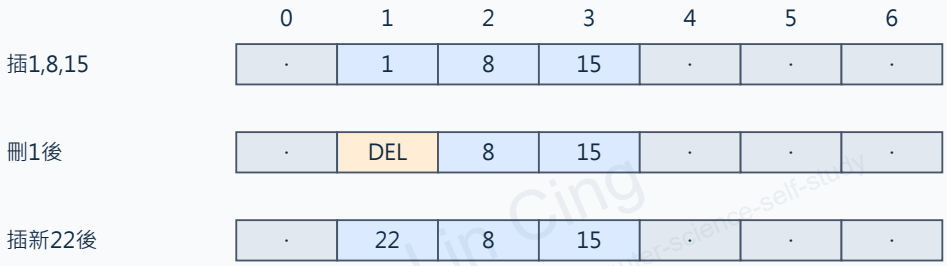
```
insert(k,value):
    first_deleted=NIL
    for i=0..m-1:
        j=h(k,i)
        if T[j] is OCCUPIED and T[j].key==k:
            update value; return j
        if T[j] is DELETED and first_deleted==NIL:
            first_deleted=j
        if T[j] is EMPTY:
            place at first_deleted if set, otherwise j; return
    if first_deleted!=NIL: place there; return
    error full
```

不能看到第一個DELETED就立即插入，否則同key可能還在後面，造成重複record。只有呼叫者已保證key全新時，才可省略後續去重掃描。Delete先search，找到後設DELETED，未找到則狀態不變。

Tombstone與不重複Key的Insert （續）

Tombstone：保住Probe路徑，也保住Key唯一性

m=7，home=k mod7，線性探測；查15沿1→2→3跨過DEL。



先插8只更新槽2；插22記住DEL槽1，探到EMPTY槽4才回填1。

灰：EMPTY可停；橘：DELETED不可停；藍：OCCUPIED需比較Key。

同一條Probe路徑如何保持可搜尋

m=7、home(k)=k mod7。依序插1、8、15，分別在槽1、2、3。刪1把槽1設DELETED；找15仍探槽1→2→3，回槽3，不能在槽1停止。

此時insert(8,new-value)會記住槽1可重用，但繼續到槽2找到8後更新，不在槽1再造一筆8。插新22時依序探1(DELETED)、2(8)、3(15)、4(EMPTY)，確認不存在才把22寫回第一個DELETED槽1。

活躍負載α=n/m下降不等於probe變短：大量tombstone會讓很低n的表仍沒有EMPTY，失敗search花m次。下面的簡潔期望界先假定無刪除/tombstone；若需套到動態工作負載，必須另外限制tombstone比例或重建並計其成本。

三種Probe公式與覆蓋條件

Linear probing：$(h_0(k)+i) \bmod m$一定覆蓋全表，但相鄰cluster會吸附後來home落在該區的key，形成primary clustering。相同home必有同probe序列。

Quadratic probing：$(h_0(k)+c_1 i+c_2 i^2) \bmod m$不會對任意參數自動覆蓋全表。例m=7、偏移$i^2$只得到0,1,4,2,2,4,1，尚有空槽也可能找不到。相同home仍同序列，稱secondary clustering。

一個可證全覆蓋的版本為$m=2^w$、偏移$f(i)=i(i+1)/2$（先整數除2再mod m）。若$f(i)=f(j) \bmod 2^w$，則2^{w+1}整除$(i-j)(i+j+1)$。兩因子一奇一偶，偶因子必被2^{w+1}整除；在$0 \le i,j<2^w$範圍，只有$i-j=0$可行，故i=j，序列為排列。例如m=8偏移0,1,3,6,2,7,5,4。

Double hashing：$(h_1(k)+i \cdot h_2(k)) \bmod m$，週期為$m/\gcd(h_2(k),m)$。寫$m=gm'$、$h_2=gd'$、$\gcd(m',d')=1$，回起點等價$m'$整除i，所以最小正週期$m'$。必須互質才覆蓋全表；m質數配非零step，或m為2冪配奇step皆可。

理想Uniform probing的失敗搜尋界

Uniform hashing在此指每key的probe permutation均勻取自m!種，且新query相對於現有表仍為均勻未偏的排列。這比chaining的一個槽均勻更強；linear/quadratic/double hashing都不能直接當成滿足此理想模型。

無tombstone、$n<m$，令X是失敗search到第一EMPTY的探測數。前$i-1$個槽都滿的機率為

$\Pr(X \ge i)=\prod_{j=0}^{i-2}(n-j)/(m-j) \le (n/m)^{i-1}=\alpha^{i-1}$，$i \le n+1$；更大的尾機率為0。

因X非負整數，$X=\sum_{i \ge 1} I[X \ge i]$，取期望得$E[X]=\sum \Pr(X \ge i) \le \sum_{i \ge 1} \alpha^{i-1}=1/(1-\alpha)$。

這是without-replacement的條件乘積，不是假設連續probe獨立。新key插入遇首EMPTY也用相同探測成本。例m=10,n=8的界為5，但不是每個query恰花5次，也不是實際linear probing的保證。

成功搜尋界必須平均插入時負載

先固定n筆相異key的插入序列，不刪除；成功query均勻選一筆。第i+1筆插入時只有i筆在表，期望probe≤m/(m−i)。後來不移動該key，也不刪除之前probe遇到的佔用槽，所以搜尋重現當時路徑。

E[成功probe] ≤ (1/n)Σ_(i=0)^(n−1) m/(m−i) = (1/α)Σ_(k=m−n+1)^m1/k

≤(1/α)∫_(m−n)^m dx/x = (1/α)ln(1/(1−α))。

積分界成立因1/x遞減，區間[k−1,k]積分≥1/k。α=0沒有成功query，不能把1/α直接代0；當α→0，右式極限為1。α→1時兩種期望界都發散，所以工程上需保留容量，但具體閾值要連同實作與成本模型決定。

11.5 Perfect hashing

Static dictionary的key集合建好後固定。FKS兩層配置讓完成建表後，每個query的最壞memory access為常數，同時使用O(n)空間；隨機重試發生在建表，不能把建表也稱最壞O(n)。

空集合直接回不存在，不做mod0。以下n≥1，第一層m=n槽，從universal族抽h，bucket j含n_j筆。令Y為無序碰撞key對數，有恆等式Σn_j²=n+2Y。每對碰撞機率≤1/n，故

E[Σn_j²] ≤ n+2·binom(n,2)/n = 2n−1。

Markov給Pr(Σn_j²≥4n)<1/2，因此重試直到Σn_j²<4n，期望少於2輪，每輪O(n)分桶。這裡不是僅說「期望空間O(n)」就宣稱某次建表已具確定線性大小；接受條件必須真的檢查。

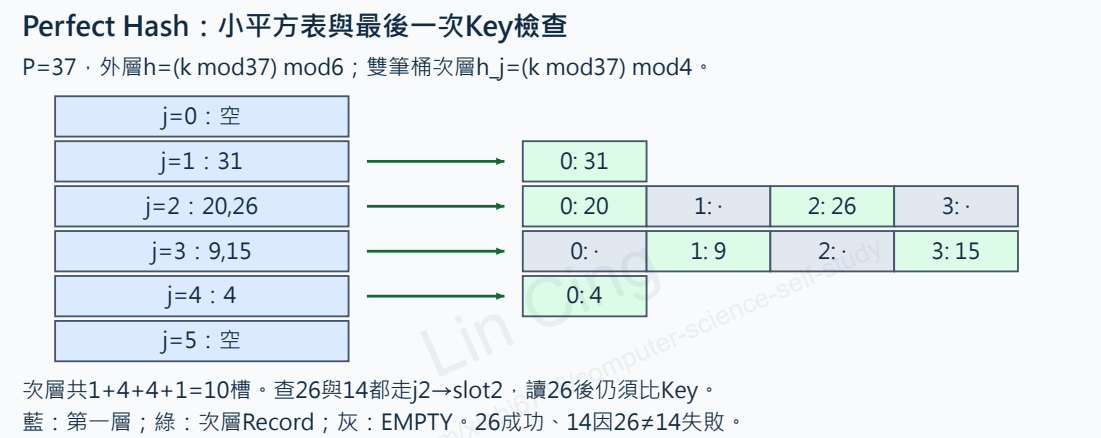
次層平方表為何很快找到無碰撞函數

每個有s=n_j≥2筆的桶配置s^2槽，另從universal族獨立抽h_j。碰撞對數Z滿足E[Z]≤binom(s,2)/s^2=(s−1)/(2s)<1/2，所以Pr(Z≥1)≤E[Z]<1/2。真檢查無碰撞才接受，期望少於2輪。

每次試驗若配置/清空s^2槽，成本O(s^2)；接受的第一層保證Σs^2<4n，故全部次層的期望總建表O(n)，連同第一層仍為O(n)。兩種隨機性分別對第一層重試與條件於已接受分桶的次層重試計，不能只看某個桶的平均就省掉總成本。

空桶不配置次層，單筆桶可直接存record，不需做mod0。Lookup先計j=h(k)，若非空再計位置h_j(k)，讀槽後仍比完整key。對不在集合的query，可能落到已占用槽，不能因兩層無碰撞就直接回答存在。

兩層表的實際配置與失敗Query



原創K={4,9,15,20,26,31}，P=37，第一層h(k)=(k mod37) mod6，桶大小[0,1,2,2,1,0]，平方槽數0+1+4+4+1+0=10<24。兩個雙筆桶用h_j(k)=(k mod37) mod4，分別[20,EMPTY,26,EMPTY]、[EMPTY,9,EMPTY,15]，無碰撞。

兩層表的實際配置與失敗Query （續）

查26：第一層2→次層2→讀26，相等所以成功。查14也走2→2，卻讀到26≠14，必回不存在。一次固定查詢只做常數個word操作和memory access，這是建表成功後的最壞保證；增加新key或更改hash參數可能破壞無碰撞性，須另重建/驗證。

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

資料來源

書目

章節來源

Cormen, Leiserson, Rivest, Stein. *Introduction to Algorithms*, 3rd ed., MIT Press, 2009，Chapter 11。

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

參考資料與署名

Thomas H. Cormen, Charles E. Leiserson, Ronald L. Rivest, Clifford Stein. Introduction to Algorithms, 3rd Edition.

Lin Cing 原創教學整理。原創內容以 CC BY 4.0 授權；參考書籍不隨教材散布。

https://github.com/xinhi6716/computer-science-self-study

Lin Cing
https://github.com/xinhi6716/computer-science-self-study